EAST WEST UNIVERSITY

Department of CSE

Course Code: CSE-246

Course Title: Algorithm

Section: 8

Fall 2024

Project Report

Project Title: Movie Management

Submitted by-

Name	Id
Jannat Milky	2023-1-60-042
Mahfuz Uddin Ahmed	2023-1-60-207

Submitted to-

Amit Mandal
Lecturer
Department of Computer Science and Engineering
East West University

Submission Date: 04-02-2025

Smart Movie Recommendations

Problem Statement:

The Smart Movie Recommendations project aims to develop an efficient system for managing and organizing a large collection of movies (up to 10,000 entries). The system is designed to handle various aspects of movie data, including movie names, durations, genres, franchises, and franchise orders. The primary objective is to provide functionalities for selecting an optimal set of movies based on a given time constraint, determining similarities between movies using their genres, and arranging selected movies in a logical order based on their franchise sequence.

The system employs 3 advanced algorithms which are **Longest Common Subsequence (LCS)** to compute genre-based similarities between movies, the **0/1 Knapsack algorithm** to select the maximum number of movies within a specified time limit, and **Topological Sorting** to arrange movies in a franchise-based order. Additionally, the project includes an auto-input generator to simulate large datasets for testing and performance evaluation. The system is implemented in C, ensuring efficient memory management and execution speed, and outputs the results to external files for easy analysis.

Implementation:

Auto input generator code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

#define MAX_MOVIES 10000

#define MAX_GENRES 10

#define MAX_NAME_LENGTH 20

#define MAX_DURATION 200
```

```
#define MAX_FRANCHISE_ORDER 10

// Arrays for random data generation

const char *GENRES[] = {"Action", "Comedy", "Drama", "Horror", "Sci-Fi", "Thriller",
"Romance", "Adventure", "Fantasy", "Mystery"};

const char *FRANCHISES[] = {"FranchiseA", "FranchiseB", "FranchiseC", "FranchiseD",
"FranchiseE"};

// Function to generate a random movie

void generate_movie(FILE *file, int index) {

    char name[MAX_NAME_LENGTH];

    int duration = 60 + rand() % (MAX_DURATION - 60 + 1); // Random duration between 60
and 200 minutes

    int genre_count = 1 + rand() % MAX_GENRES; // Random number of genres (1 to 10)

    int franchise_index = rand() % (sizeof(FRANCHISES) / sizeof(FRANCHISES[0])); //
Random franchise

    int franchise_order = 1 + rand() % MAX_FRANCHISE_ORDER; // Random franchise order
(1 to 10)

    // Generate movie name

    sprintf(name, "Movie%d", index + 1);

    // Write movie details to file

    fprintf(file, "%s %d %d ", name, duration, genre_count);
```

```
    // Write genres in the same order for every movie
    for (int i = 0; i < genre_count; i++) {
        fprintf(file, "%s ", GENRES[i % (sizeof(GENRES) / sizeof(GENRES[0]))]);
    }
    // Write franchise and franchise order
    fprintf(file, "%s %d\n", FRANCHISES[franchise_index], franchise_order);
}
int main() {
    int num_movies;
    FILE *file;
    // random number generator
    srand(time(NULL));
    // number of movies scan
    printf("Enter the number of movies to generate (100 to 10000): ");
    if (scanf("%d", &num_movies) != 1 || num_movies < 100 || num_movies > MAX_MOVIES) {
        printf("Invalid input. Please enter a number between 100 and %d.\n", MAX_MOVIES);
        return 1;
    }
    //  file opening for writing
```

```
    file = fopen("input.txt", "w");
    if (!file) {
        perror("Error opening file");
        return 1;
    }
    // Write time limit
    fprintf(file, "500\n");
    // Generate and write random movies
    for (int i = 0; i < num_movies; i++) {
        generate_movie(file, i);
    }
    // Close the file
    fclose(file);
    printf("Generated %d movies in input.txt.\n", num_movies);
    return 0;
}
```

Main Function code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

#define MAX_MOVIES 10000

#define MAX_GENRES 10

#define MAX_NAME_LENGTH 20

#define MAX_DURATION 200

#define MAX_FRANCHISE_ORDER 10

// Structure to store movie details

typedef struct {

    char name[MAX_NAME_LENGTH];

    int duration;

    char genres[MAX_GENRES][MAX_NAME_LENGTH];

    int genre_count;

    char franchise[MAX_NAME_LENGTH];

    int franchise_order;

} Movie;

// Function to compute LCS length

int lcs(char X[MAX_GENRES][MAX_NAME_LENGTH], int m, char
Y[MAX_GENRES][MAX_NAME_LENGTH], int n) {

    int **dp = malloc((m + 1) * sizeof(int *));

    for (int i = 0; i <= m; i++) {
```

```
        dp[i] = malloc((n + 1) * sizeof(int));
    }
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (strcmp(X[i - 1], Y[j - 1]) == 0)
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = (dp[i - 1][j] > dp[i][j - 1]) ? dp[i - 1][j] : dp[i][j - 1];
        }
    }
    int result = dp[m][n];
    for (int i = 0; i <= m; i++) {
        free(dp[i]);
    }
    free(dp);

    return result;
}
// Function to perform 0/1 Knapsack
int knapsack(Movie movies[], int n, int time_limit, int selected[]) {
    int **dp = malloc((n + 1) * sizeof(int *));
```

```
    for (int i = 0; i <= n; i++) {
        dp[i] = malloc((time_limit + 1) * sizeof(int));
    }
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= time_limit; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (movies[i - 1].duration <= j)
                dp[i][j] = (dp[i - 1][j] > dp[i - 1][j - movies[i - 1].duration] + 1) ? dp[i - 1][j] : dp[i - 1][j - movies[i - 1].duration] + 1;
            else
                dp[i][j] = dp[i - 1][j];
        }
    }
    //
    int remaining_time = time_limit;
    for (int i = n; i > 0; i--) {
        if (dp[i][remaining_time] != dp[i - 1][remaining_time]) {
            selected[i - 1] = 1; // Mark movie as selected
            remaining_time -= movies[i - 1].duration;
        }
    }
    int result = dp[n][time_limit];
    for (int i = 0; i <= n; i++) {
```

```
        free(dp[i]);
    }
    free(dp);

    return result;
}
// Function to perform Topological Sort
void topological_sort(Movie movies[], int n, int selected[], FILE *output_file) {
    // Create a graph representation
    int **graph = malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++) {
        graph[i] = calloc(n, sizeof(int));
    }
    int *in_degree = calloc(n, sizeof(int));

    // Build the graph based on franchise order
    for (int i = 0; i < n; i++) {
        if (!selected[i]) continue; // Skip unselected movies
        for (int j = 0; j < n; j++) {
            if (!selected[j]) continue; // Skip unselected movies
            if (strcmp(movies[i].franchise, movies[j].franchise) == 0 && movies[i].franchise_order < movies[j].franchise_order) {
                graph[i][j] = 1; // Add edge from i to j
                in_degree[j]++;
```

```
        }
    }
}
// Perform topological sort using Kahn's algorithm
int *queue = malloc(n * sizeof(int));
int front = 0, rear = 0;
// Enqueue movies with in-degree 0
for (int i = 0; i < n; i++) {
    if (selected[i] && in_degree[i] == 0) {
        queue[rear++] = i;
    }
}

// Write the sorted order to output file
fprintf(output_file, "\nTopological Order of Selected Movies:\n");
while (front < rear) {
    int u = queue[front++];
    fprintf(output_file, "%s\n", movies[u].name);
    // Reduce in-degree of adjacent movies
    for (int v = 0; v < n; v++) {
        if (graph[u][v] && selected[v]) {
            in_degree[v]--;
            if (in_degree[v] == 0) {
```

```
                queue[rear++] = v;
            }
        }
    }
    // Free allocated memory
    for (int i = 0; i < n; i++) {
        free(graph[i]);
    }
    free(graph);
    free(in_degree);
    free(queue);
}
int main() {
    Movie *movies = malloc(MAX_MOVIES * sizeof(Movie)); // Allocate on heap
    int **similarity = malloc(MAX_MOVIES * sizeof(int *)); // Allocate on heap
    for (int i = 0; i < MAX_MOVIES; i++) {
        similarity[i] = malloc(MAX_MOVIES * sizeof(int));
    }
    int *selected = calloc(MAX_MOVIES, sizeof(int)); // Allocate on heap
    int n = 0, time_limit = 0;
    clock_t start, end;
    double lcs_time, knapsack_time, topo_time;
```

```
    // Read input from file
    FILE *input_file = fopen("input.txt", "r");
    if (!input_file) {
        perror("Error opening input file");
        return 1;
    }
    // Read time limit
    fscanf(input_file, "%d", &time_limit);
    // Read movie details
    while (fscanf(input_file, "%s %d %d", movies[n].name, &movies[n].duration,
&movies[n].genre_count) != EOF) {
        for (int i = 0; i < movies[n].genre_count; i++) {
            fscanf(input_file, "%s", movies[n].genres[i]);
        }
        fscanf(input_file, "%s %d", movies[n].franchise, &movies[n].franchise_order);
        n++;
    }
    fclose(input_file);
    // Measure LCS execution time
    start = clock();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            similarity[i][j] = lcs(movies[i].genres, movies[i].genre_count, movies[j].genres,
movies[j].genre_count);
```

```
    }
}
end = clock();
lcs_time = ((double)(end - start)) / CLOCKS_PER_SEC;
// Measure Knapsack execution time
start = clock();
int max_movies = knapsack(movies, n, time_limit, selected);
end = clock();
knapsack_time = ((double)(end - start)) / CLOCKS_PER_SEC;

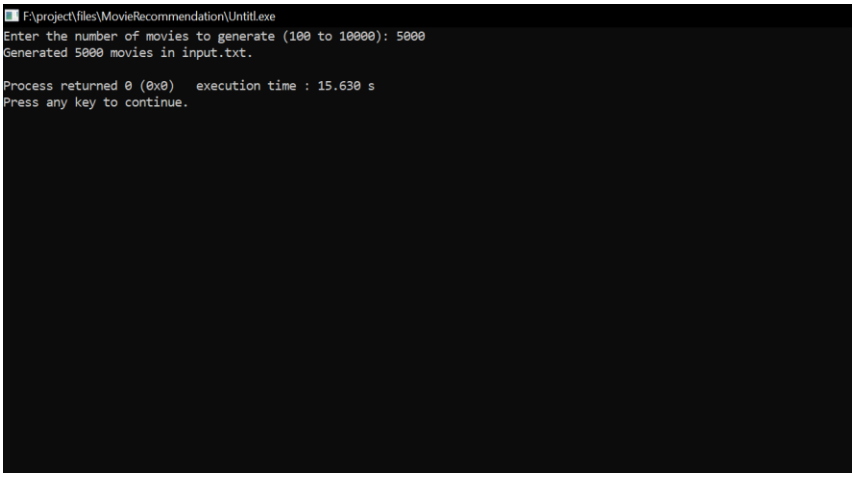
// Write output to file
FILE *output_file = fopen("output.txt", "w");
if (!output_file) {
    perror("Error opening output file");
    return 1;
}
fprintf(output_file, "Selected Movies:\n");
for (int i = 0; i < n; i++) {
    if (selected[i]) {
        fprintf(output_file, "%s\n", movies[i].name);
    }
}
// Measure Topological Sort execution time
```

```
start = clock();
topological_sort(movies, n, selected, output_file);
end = clock();
topo_time = ((double)(end - start)) / CLOCKS_PER_SEC;
fclose(output_file);
// Write execution times to file
FILE *time_file = fopen("time_results.txt", "w");
if (!time_file) {
    perror("Error opening time results file");
    return 1;
}
fprintf(time_file, "LCS Time: %f seconds\n", lcs_time);
fprintf(time_file, "Knapsack Time: %f seconds\n", knapsack_time);
fprintf(time_file, "Topological Sort Time: %f seconds\n", topo_time);
fclose(time_file);
// Free allocated memory
free(movies);
for (int i = 0; i < MAX_MOVIES; i++) {
    free(similarity[i]);
}
free(similarity);
free(selected);
printf("Program executed successfully. Check output.txt and time_results.txt for results.\n");
```

```
    return 0;

}
```

Input:



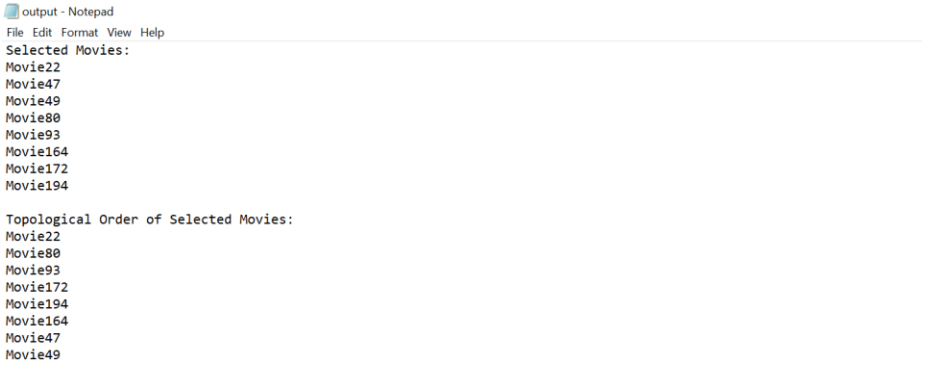
input - Notepad

File Edit Format View Help

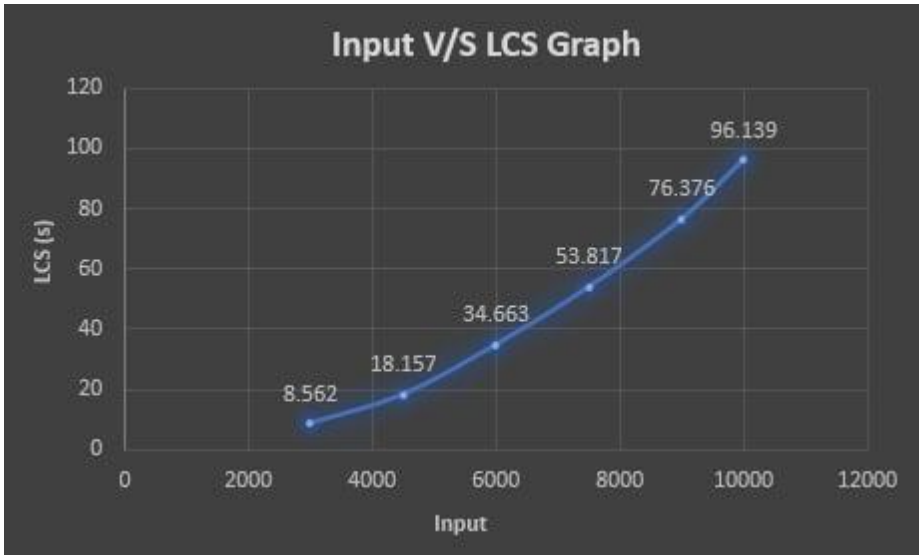
```
500
Movie1 116 3 Action Comedy Drama FranchiseA 10
Movie2 101 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseB 1
Movie3 186 2 Action Comedy FranchiseC 10
Movie4 186 5 Action Comedy Drama Horror Sci-Fi FranchiseA 1
Movie5 194 6 Action Comedy Drama Horror Sci-Fi Thriller FranchiseC 10
Movie6 130 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseD 4
Movie7 104 1 Action FranchiseE 10
Movie8 188 5 Action Comedy Drama Horror Sci-Fi FranchiseB 8
Movie9 123 6 Action Comedy Drama Horror Sci-Fi Thriller FranchiseC 9
Movie10 90 7 Action Comedy Drama Horror Sci-Fi Thriller Romance FranchiseB 6
Movie11 155 4 Action Comedy Drama Horror FranchiseC 1
Movie12 76 9 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy FranchiseA 7
Movie13 163 4 Action Comedy Drama Horror FranchiseC 7
Movie14 142 3 Action Comedy Drama FranchiseD 5
Movie15 148 8 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure FranchiseA 10
Movie16 182 9 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy FranchiseD 3
Movie17 189 2 Action Comedy FranchiseC 5
Movie18 119 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseB 1
Movie19 176 1 Action FranchiseA 7
Movie20 183 1 Action FranchiseC 2
Movie21 179 5 Action Comedy Drama Horror Sci-Fi FranchiseA 3
Movie22 63 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseB 6
Movie23 140 8 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure FranchiseB 7
Movie24 128 9 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy FranchiseC 1
Movie25 195 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseD 5
Movie26 148 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseD 6
Movie27 157 2 Action Comedy FranchiseC 8
Movie28 130 7 Action Comedy Drama Horror Sci-Fi Thriller Romance FranchiseE 8
Movie29 190 4 Action Comedy Drama Horror FranchiseC 8
Movie30 69 7 Action Comedy Drama Horror Sci-Fi Thriller Romance FranchiseC 10
Movie31 81 10 Action Comedy Drama Horror Sci-Fi Thriller Romance Adventure Fantasy Mystery FranchiseC 5
Movie32 94 6 Action Comedy Drama Horror Sci-Fi Thriller FranchiseE 5
Movie33 184 1 Action FranchiseE 6
```

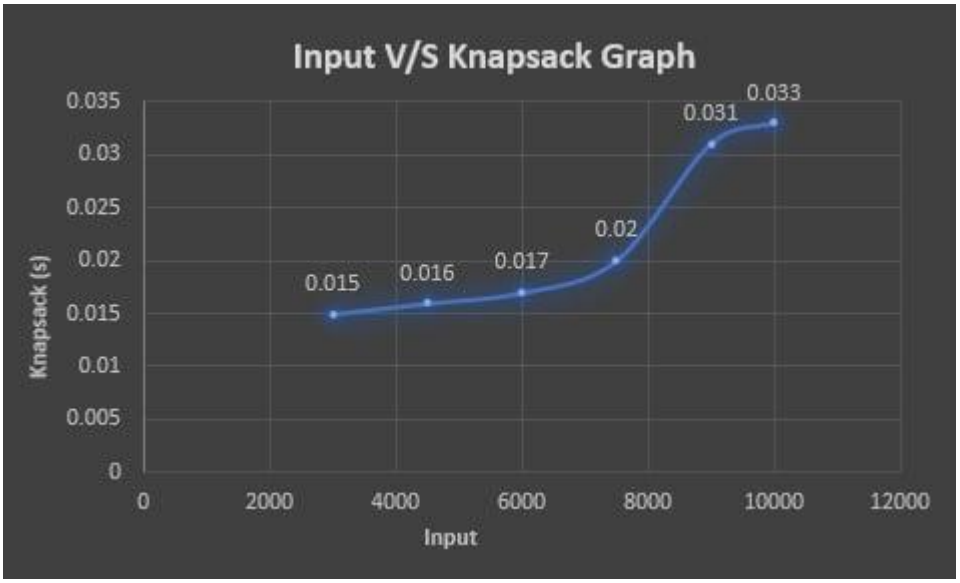
Output:

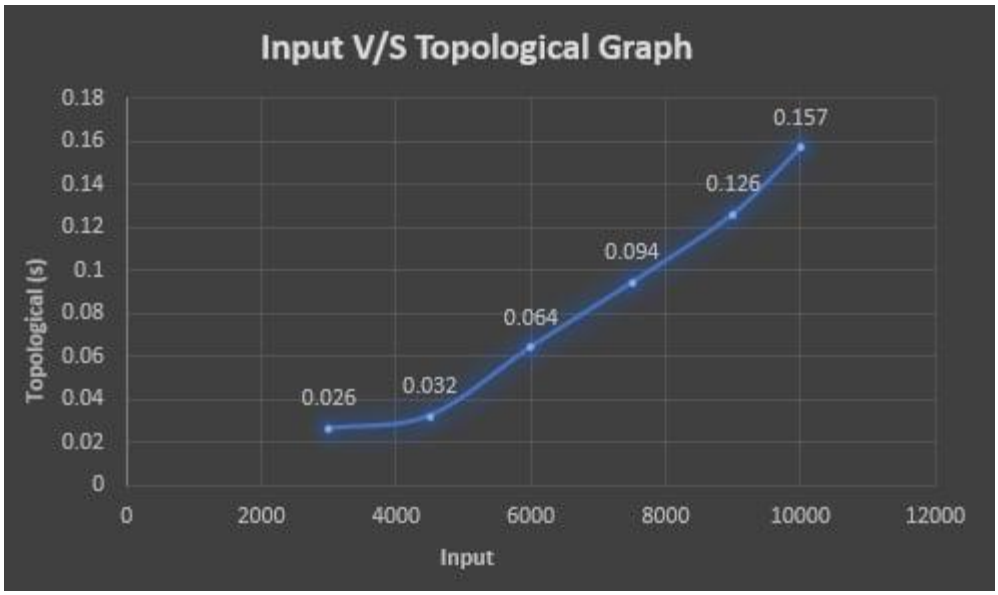

F:\project\files\MovieRecommendation\main.exe
Program executed successfully. Check output.txt and time_results.txt for results.
Process returned 0 (0x0)   execution time : 17.332 s
Press any key to continue.


output - Notepad
File Edit Format View Help
Selected Movies:
Movie22
Movie47
Movie49
Movie80
Movie93
Movie164
Movie172
Movie194
Topological Order of Selected Movies:
Movie22
Movie80
Movie93
Movie172
Movie194
Movie164
Movie47
Movie49

Graph Representation:







Discussion:

This project involves generating a list of movies, selecting a subset of movies based on a time constraint, and ordering them based on franchise dependencies. The key algorithms used are Longest Common Subsequence (LCS) for genre similarity, 0/1 Knapsack for movie selection, and Topological Sort for ordering movies within franchises.

1. Algorithm Choices

a. Longest Common Subsequence (LCS) for Genre Similarity

Purpose:

LCS is used to measure the similarity between two sequences of genres for each pair of movies. It identifies how many genres are common between two movies in the same order.This helps in understanding the similarity between movies based on their genre lists, which can be used for recommendations or clustering.

Why LCS?

LCS is a natural choice for comparing sequences, as it captures the order of genres, which is important for understanding movie similarity.It provides a clear metric for similarity, which can be used for further analysis or recommendations.

Time Complexity:

The LCS algorithm has a time complexity of O(m * n), where m and n are the lengths of the two genre sequences.In this project, since the maximum number of genres per movie is fixed

(MAX_GENRES = 10), the time complexity for comparing two movies is O(10 * 10) = O(100), which is constant. For n movies, the overall time complexity for computing LCS for all pairs is $O(n^2 * 100) = O(n^2)$.

b. 0/1 Knapsack for Movie Selection

Purpose:

The 0/1 Knapsack problem is used to select the maximum number of movies that can be watched within a given time limit (time_limit). Each movie is treated as an item with a weight (duration) and a value (1, since we want to maximize the number of movies).

Why Knapsack?

The Knapsack problem is a classic optimization problem that fits well with the requirement of maximizing the number of movies within a time constraint. It ensures an optimal solution, which is important for this use case.

Time Complexity:

The dynamic programming solution for the 0/1 Knapsack problem has a time complexity of O(n * W), where n is the number of movies and W is the time limit. In this project, W is fixed at 500, so the time complexity is O(n * 500) = O(n).

c. Topological Sort for Franchise Ordering

Purpose:

Topological sort is used to order movies within the same franchise based on their franchise_order. This ensures that movies are watched in the correct sequence within a franchise. It is particularly useful for franchises where the order of movies matters (e.g., prequels, sequels).

Why Topological Sort?

Topological sort is the standard algorithm for ordering elements based on dependencies, which aligns perfectly with the requirement of ordering movies within a franchise. It ensures that movies are watched in the correct sequence, which is critical for franchises.

Time Complexity:

The time complexity of Kahn's algorithm for topological sorting is O(V + E), where V is the number of vertices (movies) and E is the number of edges (dependencies between movies). In the worst case, E can be $O(n^2)$ if every movie depends on every other movie. However, in practice, E is much smaller since dependencies are limited to movies within the same franchise. Thus, the time complexity is O(n + E), where E is typically much smaller than n^2.

2. Best and Worst Cases

a. LCS

Best Case: All movies have the same genres in the same order. The LCS computation is trivial, but the overall complexity remains $O(n^2)$ because we still compare all pairs.

Worst Case: All movies have completely different genres. The LCS computation still takes $O(n^2)$ time.

b. Knapsack

Best Case: All movies have the same duration, and the time limit is a multiple of this duration. The Knapsack algorithm runs in O(n).

Worst Case: The time limit is large, and the durations vary significantly. The Knapsack algorithm still runs in O(n).

c. Topological Sort

Best Case: No dependencies between movies (e.g., all movies belong to different franchises). The topological sort runs in O(n).

Worst Case: All movies belong to the same franchise and have a linear dependency chain (For example, Movie1 → Movie2 → Movie3 → ...). The topological sort runs in O(n + E), where $E = O(n^2)$.

3. Challenges Faced

a. LCS

The LCS algorithm requires comparing all pairs of movies, leading to $O(n^2)$ time complexity. For large n (e.g., 10,000 movies), this becomes computationally expensive.

b. 0/1 Knapsack

The Knapsack algorithm assumes that all movies have equal value (1). This may not always be the case if some movies are more "valuable" than others.

c. Topological Sort

The topological sort assumes that the franchise order is consistent and complete. If there are missing or inconsistent franchise orders, the algorithm may fail.

4. Overall Time Complexity

LCS for Genre Similarity: $O(n^2)$

Knapsack for Movie Selection: $O(n)$

Topological Sort for Franchise Ordering: $O(n + E)$

Overall Time Complexity: $O(n^2)$ (dominated by LCS)

Conclusion:

The Movie Management System efficiently organizes and selects movies using LCS for genre-based similarity, 0/1 Knapsack for time-constrained selection, and Topological Sorting for franchise order arrangement. Implemented in C, it handles large datasets (up to 10,000 movies) with optimized performance. The system provides a practical solution for movie selection and recommendation, with potential for future enhancements like machine learning-based recommendations and a graphical interface.